

MANUAL DE USUARIO

Proxy inverso L7

Instalación, configuración y operación

Guía práctica para compilar, configurar, poner en marcha, supervisar y diagnosticar el proxy.

Versión	1.0.0
Fecha	26 de septiembre de 2026
Binario	<code>build/src/proxy</code>
Referencia	<code>proxy.toml</code> (comentado) · <code>SPEC.md</code>

Índice

1. Qué es y qué hace
2. Instalación
3. Primeros pasos (demo en 5 minutos)
4. Línea de comandos y señales
5. Referencia de configuración
 - [global]
 - [timeouts]
 - [[frontend]] y rutas
 - [[backend]] y health
6. Enrutamiento por dominio
7. HTTPS y certificados
8. Balanceo de carga
9. Health checks
10. Recarga en caliente
11. Supervisión: estadísticas y logs
12. Resolución de problemas
13. Herramientas de prueba y benchmark

1. Qué es y qué hace

El proxy se sitúa delante de tus servidores web. Escucha en uno o varios puertos (HTTP o HTTPS), mira el dominio que pide cada cliente y reenvía la petición a uno de los servidores del grupo configurado para ese dominio.

- **Varios puertos de entrada** (frontends), cada uno con sus propias reglas.
- **Enrutamiento por dominio**: exacto (`api.test`), comodín (`*.example.com`) y ruta por defecto.
- **HTTPS** con varios certificados por puerto, elegidos por el nombre que pide el cliente (SNI).
- **Balanceo**: round robin, por pesos, por menos conexiones o según la carga que informa cada servidor.
- **Tolerancia a fallos**: detecta servidores caídos y los saca del reparto.
- **Recarga sin cortes**: basta con guardar el fichero de configuración.
- HTTP/1.1 completo: conexiones persistentes, pipelining, subidas grandes y WebSocket.

Plataformas: **Linux** (verificado). macOS y BSD están soportados en el código, pero aún no se han probado en esas plataformas. Windows no está soportado.

2. Instalación

2.1 Requisitos

Para	Necesitas
Compilar	Compilador C11 (gcc o clang), Meson $\geq$ 0.60, Ninja, pkg-config, OpenSSL $\geq$ 3.0 (cabeceras)
Tests unitarios	cmocka
Tests de integración	curl, jq, socat, openssl
Benchmark	wrk

En Ubuntu/Debian:

```
sudo apt-get install build-essential meson ninja-build pkg-config libssl-dev \
    libcmocka-dev curl jq socat wrk
```

En macOS (Homebrew): `brew install meson ninja pkg-config openssl@3 cmocka wrk jq socat`.

2.2 Compilar

```
meson setup build
meson compile -C build
meson test -C build --suite unit    # opcional: 5 suites de tests
```

Se generan tres programas: `build/src/proxy` (el proxy), `build/tools/test_backend` (servidor de pruebas) y `build/tools/ws_probe` (cliente WebSocket de pruebas). Para producción, compila en modo release:

```
meson setup build-release -Dbuildtype=release -Db_lto=true
meson compile -C build-release
sudo meson install -C build-release    # instala 'proxy' en /usr/local/bin
```

3. Primeros pasos (demo en 5 minutos)

El proyecto trae un generador de configuración de prueba con su propia autoridad certificadora.

```
# 1. Configuración + certificados de prueba en ./demo
./tests/gen_config.sh demo

# 2. Unos servidores de ejemplo
./build/tools/test_backend --port 19001 --name api-1 &
./build/tools/test_backend --port 19002 --name api-2 &
./build/tools/test_backend --port 19004 --name web-1 &

# 3. El proxy
./build/src/proxy -c demo/proxy.toml
```

En otra terminal:

```
curl -H 'Host: api.test' http://127.0.0.1:18080/
hello from api-1 path=/                (+ una línea de relleno hasta 100 B)
curl -H 'Host: api.test' http://127.0.0.1:18080/
hello from api-2 path=/                (round robin)
curl -H 'Host: www.example.com' http://127.0.0.1:18080/
hello from web-1 path=/                (comodín *.example.com)

# HTTPS, verificando el certificado con la CA de la demo
curl --cacert demo/certs/ca.pem --resolve api.test:18443:127.0.0.1 https://api.test:18443/
```

Sin tocar `/etc/hosts`. `curl --resolve` o `-H 'Host: ...'` bastan para probar dominios. Si quieres usar los nombres en el navegador: `sudo ./tests/hosts.sh add` (se deshace con `remove`).

Para parar: `Ctrl+C` en la terminal del proxy. Hace una parada ordenada.

4. Línea de comandos y señales

```
proxy -c <config.toml> [-t] [-w N]
  -c fichero de configuración TOML (obligatorio)
  -t solo valida la configuración y sale
  -w fuerza el número de workers
```

Valida siempre antes de desplegar:

```
$ ./build/src/proxy -t -c demo/proxy.toml
configuración OK: 3 frontends, 8 backends, 2 workers
```

El proxy arranca un proceso **master** y N procesos **worker**. Las señales van siempre al master:

Señal	Efecto
<code>SIGHUP</code>	Recarga la configuración (también se recarga sola al guardar el fichero)
<code>SIGTERM</code> , <code>SIGINT</code>	Parada ordenada: deja de aceptar conexiones y espera hasta 10 s a las peticiones en curso

```
kill -HUP $(pgrep -o -f 'proxy -c')    # -o: el más antiguo = master
```

Si un worker muere, el master lo relanza automáticamente.

5. Referencia de configuración

La configuración es un fichero **TOML**. Las rutas relativas (certificados, log) se resuelven respecto al propio fichero. Una clave desconocida o mal escrita es un error: el proxy no arranca, o en una recarga la descarta y mantiene la configuración anterior.

5.1 [global]

Clave	Tipo	Por defecto	Descripción
<code>workers</code>	"auto" o 1-128	"auto"	Procesos worker; "auto" = nº de CPUs. Cambiarlo requiere reiniciar
<code>stats_socket</code>	ruta	/tmp/proxy.sock	Socket UNIX de estadísticas (máx. 107 caracteres)
<code>log_file</code>	ruta	stderr	Fichero de log; "-" = stderr
<code>log_level</code>	texto	"info"	debug · info · warn · error
<code>access_log</code>	booleano	true	Una línea por petición

5.2 [timeouts]

Todos en milisegundos.

Clave	Por defecto	Qué controla	Al vencer
<code>client_header</code>	10000	Tiempo máximo para recibir la cabecera (y el handshake TLS)	408
<code>client_idle</code>	60000	Conexión persistente ociosa y túnel WebSocket sin tráfico	cierre
<code>upstream_connect</code>	2000	Conexión a un servidor backend	otro servidor, o 504
<code>upstream_read</code>	30000	Tiempo sin recibir datos del backend	504
<code>upstream_idle</code>	30000	Conexión reutilizable ociosa hacia el backend	cierre

5.3 [[frontend]] y rutas

Cada `[[frontend]]` es un puerto de entrada. Sus rutas se declaran justo debajo con `[[frontend.route]]`.

Clave	Descripción
<code>name</code>	Nombre único (aparece en los logs)
<code>listen</code>	"ip:puerto", "[::]:puerto" o "*:puerto"
<code>tls</code>	true para HTTPS (requiere certs)
<code>certs</code>	Lista de { sni, cert, key } (ver capítulo 7)
<code>default_cert</code>	Índice del certificado para clientes sin SNI o con SNI desconocido (0 por defecto)
<code>route.host</code>	"dominio", "*.dominio" o "default"
<code>route.backend</code>	Nombre de un <code>[[backend]]</code>

```
[[frontend]]
name = "web"
listen = "0.0.0.0:8080"

[[frontend.route]]
host = "api.test"
backend = "api"

[[frontend.route]]
host = "default"
backend = "web"
```

5.4 [[backend]] y health

Clave	Por defecto	Descripción
name	—	Nombre único, referenciado desde las rutas
strategy	round_robin	round_robin · weighted · least_conn · least_load
max_idle	64	Conexiones persistentes ociosas que se guardan por servidor
servers	—	Lista de { addr = "host:puerto", weight = 1 } (1-64 servidores; peso 1-1000)
health.type	sin sección: none	tcp · http · none
health.path	/health	Ruta consultada con type = "http" (espera 2xx)
health.interval	2000	ms entre sondas
health.timeout	500	ms máximos por sonda
health.rise / fall	2 / 3	Éxitos para volver a "up" / fallos para pasar a "down"

```
[[backend]]
name = "api"
strategy = "least_load"
servers = [ { addr = "10.0.0.11:9000" }, { addr = "10.0.0.12:9000" } ]

[[backend.health]]
type = "http"
path = "/health"
```

6. Enrutamiento por dominio

Para cada petición se toma el dominio de la cabecera `Host` (sin puerto, sin distinguir mayúsculas) y se busca en las rutas del frontend por el que entró, en este orden:

1. **Dominio exacto:** `api.test`.
2. **Comodín:** `*.example.com` cubre `a.example.com` y `a.b.example.com`, pero no `example.com`. Si varios encajan, gana el más específico (`*.eu.example.com` antes que `*.example.com`).
3. **default**, si existe.
4. Si nada encaja: **404**.

Host pedido	Rutas: <code>api.test</code> · <code>*.example.com</code> · <code>default</code>
<code>api.test</code> , <code>API.TEST:8080</code>	<code>api.test</code>
<code>www.example.com</code>	<code>*.example.com</code>
<code>example.com</code> , <code>otro.test</code>	<code>default</code> (sin default: 404)

Cada petición de una conexión persistente se enruta por separado. El backend recibe además `X-Forwarded-For`, `X-Forwarded-Proto` (`http` / `https`) y `X-Real-IP` con la IP del cliente.

7. HTTPS y certificados

Un frontend con `tls = true` termina HTTPS (TLS 1.2 y 1.3); hacia los backends el tráfico va en HTTP normal. Puedes poner varios certificados en el mismo puerto: se elige según el nombre (SNI) que envía el cliente.

```
[[frontend]]
name = "web-tls"
listen = "0.0.0.0:443"
tls = true
certs = [
  { sni = "api.test", cert = "certs/api.crt", key = "certs/api.key" },
  { sni = "*.example.com", cert = "certs/wild.crt", key = "certs/wild.key" },
]
default_cert = 0
```

- `cert` : certificado en PEM; puede incluir la cadena intermedia a continuación.
- `key` : clave privada en PEM, sin contraseña. Se comprueba que corresponde al certificado.
- En `sni` , el comodín cubre una sola etiqueta (`*.example.com` no vale para `a.b.example.com`), igual que en los certificados reales.

Si el cliente pide por SNI un nombre y luego envía un `Host` distinto que su certificado no cubre, recibe **421 Misdirected Request**.

Crear un certificado de prueba

```
openssl req -x509 -newkey rsa:2048 -nodes -days 365 -sha256 \
  -keyout certs/api.key -out certs/api.crt \
  -subj "/CN=api.test" -addext "subjectAltName=DNS:api.test"
```

Comprobar qué certificado sirve el proxy:

```
openssl s_client -connect 127.0.0.1:443 -servername api.test </dev/null 2>/dev/null \
| openssl x509 -noout -subject
```

8. Balanceo de carga

Estrategia	Cómo reparte	Úsala cuando...
<code>round_robin</code>	Por turnos	Los servidores son iguales y las peticiones parecidas
<code>weighted</code>	Proporcional a <code>weight</code> , sin ráfagas	Hay máquinas de distinta capacidad
<code>least_conn</code>	Al que tiene menos peticiones en curso	Algunas peticiones son mucho más lentas que otras
<code>least_load</code>	Al que informa de menos carga	Los servidores pueden medir su propia carga

Cómo informa un servidor de su carga

Con `least_load`, cada servidor añade a sus respuestas la cabecera `X-Backend-Load` con un número de **0.0** (libre) a **1.0** (saturado). El proxy la lee, la suaviza para evitar saltos bruscos y **la elimina** antes de responder al cliente.

```
HTTP/1.1 200 OK
Content-Type: application/json
X-Backend-Load: 0.37
```

Si un servidor deja de enviarla durante más de 5 s, su carga se considera desconocida y se reparte por conexiones activas.

El reparto es por worker: con 8 workers, cada uno aplica la estrategia a sus propias peticiones, y el efecto global es el mismo.

9. Health checks

- **Pasivos** (siempre activos): si conectar a un servidor falla `fall` veces seguidas, deja de recibir tráfico. Sin sondas configuradas, se vuelve a probar pasado `interval`.
- **Activos** (sección `[backend.health]`): el proxy consulta cada servidor cada `interval` ms, por TCP o con `GET path` esperando 2xx. Tras `fall` fallos pasa a "down"; tras `rise` éxitos vuelve a "up".

Cuando un servidor falla al conectar, la petición se reintenta en otro sin que el cliente lo note. Si no queda ninguno sano, el cliente recibe **503**. Los cambios de estado quedan en el log:

```
[warn] [w0] backend 'api' servidor 127.0.0.1:9002 -> DOWN (sonda http)
[warn] [w0] backend 'api' servidor 127.0.0.1:9002 -> UP (sonda http)
```

10. Recarga en caliente

Para cambiar la configuración, **edita y guarda el fichero**: el proxy lo detecta en menos de un segundo (funciona igual con editores que guardan por renombrado, como vim, o con `sed -i`). También puedes forzarla con `kill -HUP`.

- La nueva configuración se valida entera antes de aplicarse. Si tiene un error, **se descarta, se mantiene la actual** y el motivo queda en el log.
- Las peticiones en curso terminan con la configuración anterior; las nuevas usan la nueva. No se corta ninguna conexión.
- Se pueden añadir o quitar frontends (puertos), rutas, backends, servidores y certificados.
- Cambiar `workers` o `stats_socket` requiere reiniciar.

```
[info] [master] configuración recargada (cambio en fichero)
[error] [master] recarga rechazada, se mantiene la configuración actual:
                TOML inválido: line 133: syntax error
```


12. Resolución de problemas

Síntoma	Causa probable	Qué hacer
No arranca: <code>clave desconocida 'x'</code>	Errata en la configuración	Corrige el nombre; valida con <code>proxy -t -c ...</code>
No arranca: <code>bind ...: Address already in use</code>	Otro programa usa ese puerto	<code>ss -ltnp grep :8080</code>
No arranca: no se puede cargar el certificado	Ruta mal escrita (es relativa al <code>.toml</code>) o fichero ilegible	Revisa rutas y permisos
la clave no corresponde al certificado	Pareja cert/key equivocada	Revisa la pareja cert/key
Log: <code>stats_socket</code> demasiado largo	Ruta del socket de más de 107 caracteres	Usa una ruta corta, p. ej. <code>/tmp/proxy.sock</code>
404	Ninguna ruta para ese dominio	Añade la ruta o una <code>default</code> ; revisa el <code>Host</code> que envía el cliente
421	SNI y Host no coinciden	Usa el mismo nombre en la URL y en <code>Host</code> , o un certificado que cubra ambos
431	Cabeceras de más de 16 KB (cookies enormes)	Reduce las cabeceras del cliente
502	No se pudo conectar a ningún servidor, o respondió mal	Comprueba que los backends escuchan en <code>addr</code> ; mira <code>failures</code> en las estadísticas
503	Todos los servidores del backend están "down"	Revisa las sondas (<code>path</code> , puerto) y el log de cambios de estado
504	El backend tardó más que <code>upstream_read</code>	Sube el timeout o investiga el backend
408	El cliente no terminó de enviar la cabecera a tiempo	Normal con clientes lentos o escáneres; ajusta <code>client_header</code>
Un cambio de config no se aplica	La recarga se rechazó	Busca <code>recarga rechazada</code> en el log; <code>reloads_failed</code> en las estadísticas

13. Herramientas de prueba y benchmark

13.1 Servidor de pruebas: test_backend

```
test_backend --port N --name NOMBRE [--latency-ms N] [--load F | --load-auto]
              [--threads N] [--bind IP] [--drop-every N]
```

Ruta	Respuesta
/ (cualquier otra)	hello from NOMBRE path=... más relleno hasta 100 B
/health	200 "ok"
/echo	Devuelve el cuerpo enviado (también chunked)
/headers	Devuelve las cabeceras recibidas; útil para ver X-Forwarded-*
/sleep/MS , /big/N	Responde tras MS milisegundos / cuerpo de N bytes
/chunked , /close , /nocontent	Respuesta chunked / delimitada por cierre / 204
/ws	WebSocket de eco
/stats	JSON con las conexiones y peticiones recibidas

--load 0.3 fija la cabecera X-Backend-Load ; --load-auto la calcula según las conexiones activas.

13.2 Cliente WebSocket: ws_probe

```
ws_probe --host api.test 127.0.0.1 18080 /ws "hola"
ws_probe --tls --ca demo/certs/ca.pem --sni api.test 127.0.0.1 18443 /ws "hola"
```

13.3 Tests automáticos

```
meson test -C build --suite unit # 31 tests unitarios
./tests/integration/run.sh        # 96 comprobaciones de extremo a extremo (~20 s)
```

El script de integración genera su propia configuración, lanza 12 servidores y el proxy, comprueba todas las funcionalidades y lo limpia todo al terminar. PORT_BASE=N cambia los puertos y KEEP_WORK=1 conserva los ficheros para inspeccionarlos.

13.4 Benchmark

```
./bench/bench_proxy.sh # 30 s por escenario
DURATION=10s WORKERS=4 ./bench/bench_proxy.sh # variante rápida
```

El script compila en release, reparte CPUs entre wrk, el proxy y los backends, y deja un informe en bench/results/. Resultado de referencia (8 workers, WSL2, Ryzen 7 7735HS): **223.705 req/s por HTTPS** con 100 conexiones y 337.904 por HTTP, sin errores.